دليل التصعيد الأفقي إلى الرأسي للصلاحيات

منهجية متقدمة للانتقال من حساب مستخدم عادي (Horizontal) إلى السيطرة الكاملة للمدير (Vertical Privilege Escalation)

1. تصنيف ثغرات التصعيد ومصفوفة الخطورة (OWASP A01)

يحدث التصعيد الأفقي إلى الرأسي عندما ينجح المهاجم الذي لديه صلاحيات مستخدم عادي في استغلال عيوب منطق التحكم في الوصول (Access Control) للوصول إلى وظائف وسارات المشرفين (Admin Functions).

مستوى الخطورة	المتجه المستهدف (Target Vector)	آلية الاستغلال والتحكم
P1 - حرج جداً	السيطرة الإدارية الكاملة (Admin Takeover)	تعديل معاملات الدور (Role Parameter) أثناء طلبات تحديث الحساب أو استغلال واجهات برمجة التطبيقات غير المحمية (APIs).
P2 - عالي	الوصول لوحادة تحكم المشرفين (Admin Panel Access)	تخطي حواجز واجهة المشرف (Forced Browsing) عبر مسارات مخفية مثل `admin/dashboard/` أو `internal/manage/`.
P3 - متوسط	تنفيذ وظائف إدارية (Action Tampering)	إرسال طلبات POST مباشرة لوظائف مخصصة للمديرين مثل حذف المستخدمين أو تغيير الصلاحيات باستخدام حساب عادي.

2. سلاسل الهجوم المتقدمة (Advanced Attack Chains)

- **السلسلة الأولى (Mass Assignment):** استغلال حقول مخفية في طلبات التحديث (PUT /api/users/profile) بإضافة حقل `role":"admin""` أو `is_admin": true"` لترقية الحساب تلقائياً.
- **السلسلة الثانية (Function-Level Access Bypass):** الوصول المباشر لروابط الإدارة المخفية (Hidden Admin Endpoints) عبر اكتشاف مسارات الـ JavaScript أو ملفات `robots.txt`.
- **السلسلة الثالثة (Parameter Pollution & Method Tampering):** تغيير نوع الطلب من GET إلى POST أو حقن معاملات إضافية لتضليل محرك التحقق من الصلاحيات.

3. أمثلة حية لطلبات الاستغلال (Payloads & Requests)

حقن معامل الدور (Role Injection) عبر طلب التحديث:

```
POST /api/v1/user/update-profile HTTP/1.1
Host: target-app.com
Authorization: Bearer LOW_PRIV_USER_TOKEN
Content-Type: application/json

}
"user_id": 1042,
"username": "normal_user",
"role": "admin",
"permissions": ["all"]
{
```

دليل التصعيد الأفقي إلى الرأسي (تابع)

المعالجة الآمنة والأكواد البرمجية الحامية وقائمة التحقق الميدانية

4. المعالجة الآمنة والدفاع البرمجي (Secure Remediation)

تنفيذ خاطئ (صلاحيات ضمنية يعتمد على المدخلات):

```
@PostMapping("/api/user/settings")
public ResponseEntity updateSettings(@RequestBody UserProfileForm form) {
// خطأ فادح: الثقة العمياء بالبيانات المرسلة من العميل بما فيها الدور والصلاحيات
userService.updateWithRole(form.getUserId(), form.getRole(), form.getData());
return ResponseEntity.ok().build();
}
```

تنفيذ آمن (التحقق الصارم من سياق الجلسة والصلاحيات):

```
@PostMapping("/api/user/settings")
public ResponseEntity updateSettings(@RequestBody UserProfileForm form, Authentication auth) {
// أمان تام: جلب الهوية والصلاحيات حصرياً من سياق الجلسة الموثوقة للمستخدم الحالي
UserPrincipal principal = (UserPrincipal) auth.getPrincipal();
userService.updateUserData(principal.getId(), form.getData());
return ResponseEntity.ok().build();
}
```

5. قائمة التحقق الميدانية للاختبار (Verification Checklist)

- ☐ اختبار إضافة حقول الصلاحيات والأدوار (Role, IsAdmin) في جميع طلبات التحديث (PUT / PATCH / POST).
- ☐ فحص الوصول المباشر لجميع واجهات وبرمجيات المشرفين باستخدام توكن مستخدم عادي.
- ☐ التحقق من التغيير في طرق الطلب (HTTP Method Tampering) مثل تجربة POST بدلاً من GET على مسارات الإدارة.
- ☐ مراجعة الـ Headers المخصصة (مثل X-Original-URL أو X-Admin) واختبار تأثيرها على تخطي الصلاحيات.